

SIMULADO 2/8 — GIT E GITHUB

40 QUESTÕES — 75 PONTOS

Pesos 1, 2 e 3 | Tempo sugerido: 3 horas

INSTRUÇÕES:

- Este simulado contém 40 questões de múltipla escolha (A, B, C, D, E) elaboradas rigorosamente a partir do material da Apple Developer Academy fornecido.
- As questões possuem pesos variados (1, 2 e 3) e testarão comandos, tomadas de decisão e interpretação do estado do repositório.
- Apresentarei **uma questão por vez**.
- Digite apenas a letra da alternativa que considerar correta (ou faça comentários para si mesmo), e aguarde. Apresentarei a próxima questão logo em seguida.
- Não revelarei o gabarito, não darei dicas e não informarei se você acertou ou errou ao longo do teste, reproduzindo as condições reais de prova.
- Quando quiser encerrar a aplicação e gerar o relatório detalhado de correção, domínio e análise de erros, digite: **FINALIZAR PROVA**.

QUESTÃO 1 (Peso 2)

Um desenvolvedor está trabalhando em um projeto localmente. Ele edita um arquivo já conhecido pelo Git chamado `script.js` e executa o comando `git add script.js`. Logo em seguida, ele abre o arquivo `script.js` novamente, adiciona uma nova linha de código e salva a alteração, mas **não** executa nenhum outro comando no terminal.

Considerando a mecânica das três árvores (Working Directory, Staging Area e Repository), ao executar `git status` neste momento, qual é a interpretação correta do estado do repositório?

- A) O arquivo aparecerá exclusivamente como *staged* (preparado), pois o `git add` cria um vínculo dinâmico que envia todas as alterações subsequentes para a Staging Area automaticamente.
- B) O arquivo aparecerá simultaneamente como *staged* e *modified*, revelando que o Git trabalha com *snapshots* — a versão preparada no índice foi preservada, mas o diretório de trabalho já avançou para um novo estado.
- C) O arquivo aparecerá apenas como *modified*, pois qualquer edição no Working Directory sobrescreve e anula automaticamente o conteúdo que havia sido enviado para a Staging Area.
- D) O arquivo retornará ao estado *untracked*, pois a divergência entre a Staging Area e o Working Directory quebra o rastreamento, exigindo um novo `git init`.

E) O Git indicará um erro de conflito local, pois é proibido modificar um arquivo que já foi preparado para o próximo *commit* sem antes usar o comando `git restore --staged`.

QUESTÃO 2 (Peso 1)

O arquivo `.gitignore` é utilizado para especificar arquivos não rastreados que o Git deve ignorar. Se um desenvolvedor adicionar o arquivo `config.json` ao `.gitignore`, mas esse arquivo já tiver sido rastreado e commitado anteriormente no repositório, o que acontecerá?

A) O Git apagará o arquivo `config.json` do disco local na próxima vez que `git commit` for executado.

B) O Git ignorará as futuras modificações em `config.json`, tratando-o imediatamente como invisível e imutável.

C) O Git continuará rastreando o arquivo `config.json`. É necessário removê-lo do cache do Git para que a regra do `.gitignore` tenha efeito.

D) O Git emitirá um erro de sintaxe, impedindo novos commits até que o arquivo seja removido do `.gitignore`.

E) O `.gitignore` sobrescreve o histórico, removendo o `config.json` de todos os commits anteriores automaticamente.

QUESTÃO 3 (Peso 2)

Ao colaborar em um repositório remoto, a sincronização do código é fundamental. Qual alternativa define corretamente a diferença técnica entre os comandos `git fetch` e `git pull`?

A) `git fetch` baixa as alterações do repositório remoto para o local e as mescla automaticamente com a branch atual, enquanto `git pull` apenas faz o download.

B) `git pull` é o equivalente a executar `git fetch` seguido de `git merge`, pois ele baixa as atualizações remotas e tenta integrá-las à branch atual.

C) Ambos executam exatamente a mesma ação, mas `git fetch` é usado para repositórios locais e `git pull` para repositórios hospedados no GitHub.

D) `git fetch` envia as alterações locais para o repositório remoto, enquanto `git pull` traz as alterações remotas para o ambiente local.

E) `git pull` atualiza apenas os arquivos na Staging Area, enquanto o `git fetch` atualiza diretamente o Working Directory.

QUESTÃO 4 (Peso 3)

Um desenvolvedor precisa inspecionar um código antigo e executa o comando `git checkout a1b2c3d` (sendo `a1b2c3d` o hash de um commit). O terminal exibe um aviso de "Detached HEAD" (HEAD desanexada). Qual é o significado técnico desse estado?

- A) O repositório local perdeu a conexão com o repositório remoto e precisa de uma nova configuração de URL.
- B) O ponteiro HEAD está apontando diretamente para um commit específico em vez de apontar para a referência do topo de uma branch.
- C) A branch atual foi deletada acidentalmente e o Git está executando uma rotina de recuperação.
- D) O Git separou a Staging Area do Working Directory, impedindo que novos arquivos sejam rastreados.
- E) O ponteiro HEAD foi corrompido e é necessário usar `git reset --hard` para restaurar o fluxo normal.

QUESTÃO 5 (Peso 2)

Durante um `git merge` entre a branch `feature` e a branch `main`, ocorre um conflito de mesclagem no arquivo `app.c`. Qual é o procedimento correto e padrão no Git para resolver esse conflito e concluir o merge?

- A) Apagar a branch `feature`, recriá-la a partir da `main` e reescrever o código conflitante do zero.
- B) Usar o comando `git resolve app.c`, que escolhe automaticamente a versão mais recente com base no timestamp do sistema operacional.
- C) Abrir o arquivo `app.c`, editar manualmente as seções marcadas pelo Git, salvar o arquivo, executar `git add app.c` e, em seguida, `git commit`.
- D) Executar `git push --force` para obrigar o repositório a aceitar a versão local e sobrescrever os dados conflitantes automaticamente.
- E) O conflito é resolvido rodando `git commit -am "Resolve"`, que aceita todas as alterações da branch de destino e ignora a branch de origem.

QUESTÃO 6 (Peso 1)

No fluxo de trabalho do GitHub, o que caracteriza corretamente um "Pull Request" (PR)?

- A) É um comando de terminal usado para forçar o download de arquivos de um repositório remoto.

- B) É uma ferramenta de colaboração usada para solicitar que os mantenedores de um repositório analisem e aprovem a integração das alterações de uma branch para outra.
- C) É um tipo de branch especial que só permite leitura dos arquivos, impedindo novos commits.
- D) É o processo automático onde o GitHub deleta branches inativas após a aprovação do código.
- E) É a ação de clonar um repositório de outra pessoa para a sua própria conta do GitHub.

QUESTÃO 7 (Peso 3)

Você criou um commit, mas percebeu que esqueceu de incluir um arquivo modificado que deveria fazer parte dele. O commit ainda não foi enviado ao repositório remoto (`git push`). Como você pode adicionar este arquivo ao commit anterior sem criar um commit extra no histórico?

- A) Executar `git add <arquivo>` seguido de `git reset --hard HEAD~1`.
- B) Executar `git add <arquivo>` seguido de `git commit --amend`.
- C) Executar `git restore --staged <arquivo>`.
- D) Executar `git commit -m "update"` e depois `git rebase -i`.
- E) Executar `git stash push <arquivo>`.

QUESTÃO 8 (Peso 2)

No contexto de armazenamento temporário de mudanças (Stash), qual a principal diferença entre os comandos `git stash pop` e `git stash apply`?

- A) `pop` aplica as mudanças salvas e as remove imediatamente da lista do stash; `apply` aplica as mudanças, mas as mantém armazenadas na lista.
- B) `pop` exclui as mudanças do stash sem aplicá-las no Working Directory; `apply` aplica as mudanças e as remove da lista.
- C) `pop` só funciona se o Working Directory estiver limpo; `apply` funciona mesmo com conflitos iminentes.
- D) `pop` é usado para transferir stashes entre branches diferentes; `apply` é usado estritamente na mesma branch.

E) Não há diferença prática; ambos executam internamente o comando `git restore --source=stash`.

QUESTÃO 9 (Peso 3)

Sobre a diferença conceitual e estrutural entre `git merge` e `git rebase` na integração de branches, é correto afirmar:

- A) O `merge` reescreve o histórico colocando os commits da branch atual antes dos da branch alvo; o `rebase` cria um novo commit de junção (merge commit).
- B) O `rebase` move os commits da sua branch atual para o topo da branch de destino, criando um histórico linear limpo; o `merge` preserva a divergência original criando um commit de mesclagem estrutural.
- C) O `merge` é uma ação local que só pode ser desfeita com reflog, enquanto o `rebase` altera permanentemente os arquivos no servidor remoto no momento da execução.
- D) O `rebase` é o padrão de integração exigido ao clonar projetos open-source, enquanto o `merge` é restrito a repositórios privados.
- E) O `merge` exige resolução manual de conflitos arquivo por arquivo; o `rebase` resolve todos os conflitos automaticamente usando inteligência algorítmica do Git.

QUESTÃO 10 (Peso 2)

Um projeto possui as branches `main` e `feature-login`. Você está atualmente na branch `main` e deseja trazer todas as atualizações já finalizadas da `feature-login` para a `main`. Qual é a sequência de comandos locais apropriada?

- A) `git checkout feature-login` seguido de `git merge main`
- B) `git switch feature-login` seguido de `git pull main`
- C) Estando na `main`, executar `git merge feature-login`
- D) `git branch -d feature-login` seguido de `git rebase main`
- E) `git fetch feature-login` seguido de `git commit`

QUESTÃO 11 (Peso 1)

O comando `git clone <url>` executa as seguintes operações em sequência:

- A) Inicializa um novo repositório local (`init`), puxa os dados remotos, cria uma cópia da branch remota padrão e adiciona o repositório original como um remote chamado `origin`.

- B) Baixa o código-fonte compactado em um arquivo temporário e o descompacta, sem inicializar a pasta oculta `.git`.
- C) Deleta o repositório local existente e transfere os arquivos da URL permanentemente, impedindo futuras conexões.
- D) Configura o nome e email do usuário globalmente e cria automaticamente um Fork do repositório no GitHub do usuário.
- E) Executa um `git fetch` global e sincroniza todas as branches existentes na máquina com os servidores da URL fornecida.

QUESTÃO 12 (Peso 3)

Um desenvolvedor executou acidentalmente `git reset --hard HEAD~1` na sua branch, perdendo o commit mais recente que ainda não tinha feito push para o GitHub. Ele deseja recuperar esse commit. Qual ferramenta do Git registra as atualizações da ponta das branches e permite encontrar o hash desse commit perdido?

- A) `git status`
- B) `git reflog`
- C) `git diff --cached`
- D) `git log --oneline`
- E) `git fsck --lost-found`

QUESTÃO 13 (Peso 2)

Em projetos de código aberto no GitHub, o conceito de "Fork" é o pilar da colaboração externa. Qual a definição correta de um Fork?

- A) É um comando interno do Git para dividir o histórico de um repositório grande em módulos independentes.
- B) É uma cópia integral de um repositório feita diretamente na sua conta do GitHub, permitindo experimentar e alterar o código livremente sem afetar o projeto original.
- C) É o processo de clonar um repositório para o disco rígido usando chave de segurança SSH ao invés de HTTPS.
- D) É a ferramenta do Git usada para forçar a renomeação da branch primária de `master` para `main`.
- E) É um alerta do GitHub informando que o repositório sofreu uma ramificação não autorizada.

QUESTÃO 14 (Peso 2)

Qual comando é utilizado para listar no terminal todos os repositórios remotos atualmente configurados no seu projeto local, exibindo as URLs associadas de *fetch* e *push*?

- A) `git remote -v`
- B) `git origin ls`
- C) `git fetch --all`
- D) `git config --list-remotes`
- E) `git show-branch --remotes`

QUESTÃO 15 (Peso 3)

Considere que um projeto possui a branch `main` e a branch `experimento`. Você, trabalhando na `main`, descobre que o commit de hash `7a9b2c3`, que está isolado no meio da branch `experimento`, corrige uma falha crítica de interface. Você precisa aplicar **apenas** as mudanças desse commit específico na `main`, sem integrar o restante da branch. Qual comando resolve esse problema?

- A) `git merge 7a9b2c3`
- B) `git rebase experimento`
- C) `git cherry-pick 7a9b2c3`
- D) `git restore --source=7a9b2c3`
- E) `git apply 7a9b2c3`

QUESTÃO 16 (Peso 1)

Qual é a principal função do comando `git log --oneline` em comparação ao `git log` padrão?

- A) Ocultar commits feitos por outros usuários e mostrar apenas os seus.
- B) Condensar a exibição do histórico, mostrando apenas os primeiros caracteres do hash e a mensagem do commit em uma única linha.
- C) Exibir apenas o commit mais recente do repositório em formato reduzido.
- D) Agrupar todos os commits de uma branch em um único commit visual.
- E) Exibir as alterações de código (diff) linha por linha de cada commit.

QUESTÃO 17 (Peso 2)

Um desenvolvedor introduziu um bug no sistema no commit `d4f8g9h` e já enviou isso para o repositório remoto. Ele precisa desfazer essa alteração sem reescrever o histórico público, para não prejudicar outros desenvolvedores. Qual é a abordagem correta?

- A) `git reset --hard d4f8g9h`
- B) `git rebase -i d4f8g9h`
- C) `git revert d4f8g9h`
- D) `git checkout d4f8g9h -- force`
- E) `git clean -f d4f8g9h`

QUESTÃO 18 (Peso 2)

Você deletou a branch `feature-x` localmente usando `git branch -d feature-x`. No entanto, a branch ainda existe no repositório remoto no GitHub. Qual comando remove a branch do repositório remoto?

- A) `git remote rm feature-x`
- B) `git push origin --delete feature-x`
- C) `git fetch origin --prune feature-x`
- D) `git branch -D origin/feature-x`
- E) `git pull --remove feature-x`

QUESTÃO 19 (Peso 1)

No Git, qual é a principal finalidade das Tags (ex: `git tag v1.0.0`)?

- A) Marcar arquivos específicos que não devem ser modificados no próximo commit.
- B) Criar referências permanentes para pontos específicos no histórico, geralmente usadas para demarcar versões de lançamento (releases).
- C) Adicionar metadados de busca para o GitHub indexar o repositório mais rapidamente.
- D) Substituir o uso de branches em projetos de longo prazo.
- E) Identificar conflitos de merge automaticamente.

QUESTÃO 20 (Peso 2)

Se você modificar um arquivo rastreado no Working Directory, mas ainda não o adicionou à Staging Area com `git add`, qual comando permite visualizar as linhas exatas que foram alteradas nesse arquivo?

- A) `git diff g`
- B) `git diff --staged`
- C) `git status -v`
- D) `git log -p`
- E) `git show HEAD`

QUESTÃO 21 (Peso 1)

O Git rastreia arquivos, não diretórios vazios. Se um desenvolvedor precisar obrigatoriamente manter a estrutura de uma pasta vazia no repositório remoto, qual é a convenção padrão adotada pela comunidade? A) Configurar o `.gitignore` com a flag `!empty_dir`. B) Criar um arquivo oculto e vazio dentro da pasta, frequentemente nomeado como `.gitkeep`. C) Usar o comando `git add <diretorio> --force-empty`. D) Alterar as configurações do Git global para `core.trackEmptyDirs = true`. E) Criar um submódulo do Git apenas para aquele diretório.

QUESTÃO 22 (Peso 3)

A respeito das variações do comando `git reset (--soft, --mixed, --hard)`, é correto afirmar que: A) O `--soft` remove os commits e apaga os arquivos alterados do disco; o `--hard` mantém os arquivos na Staging Area. B) O `--mixed` (padrão) desfaz o commit e remove as alterações da Staging Area, mas as preserva no Working Directory; o `--hard` descarta as alterações de ambos. C) O `--soft` desfaz o commit e move as alterações para o stash; o `--mixed` altera o histórico remoto imediatamente. D) O `--hard` é a única opção reversível nativamente sem o uso do reflog. E) O `--soft` e o `--mixed` possuem o mesmo comportamento se o repositório possuir apenas uma branch.

QUESTÃO 23 (Peso 1)

No GitHub, qual é a função primária das "Issues"? A) Fornecer um terminal online embutido para executar comandos Git. B) Servir como um sistema nativo de rastreamento de tarefas, bugs, melhorias e discussões estruturadas no repositório. C) Bloquear commits que contenham falhas de segurança detectadas automaticamente. D) Gerenciar a cobrança financeira e os limites de armazenamento do projeto. E) Substituir a necessidade de escrever mensagens de commit detalhadas.

QUESTÃO 24 (Peso 2)

Você fez um Fork de um repositório open-source e clonou o seu Fork para a máquina local. O repositório original continuou recebendo atualizações. Como você adiciona uma referência ao repositório original para puxar essas novas atualizações? A) `git clone <url-original> --update` B) `git fetch --all-forks` C) `git remote add upstream <url-original>` D) `git config global fork.source <url-original>` E) `git pull origin-master <url-original>`

QUESTÃO 25 (Peso 2)

O comando `git commit -a -m "mensagem"` agiliza o fluxo de trabalho ao pular a etapa explícita do `git add`. No entanto, qual é a principal limitação desse comando? A) Ele não funciona se houver mais de um arquivo modificado. B) Ele adiciona à Staging Area arquivos modificados e deletados, mas ignora completamente arquivos novos (untracked) que nunca foram adicionados antes. C) Ele só pode ser executado na branch `main`. D) Ele comita apenas os arquivos que estão no diretório raiz, ignorando subpastas. E) Ele falha caso haja qualquer arquivo listado no `.gitignore`.

QUESTÃO 26 (Peso 1)

Para que os seus commits sejam registrados com o seu nome e endereço de e-mail corretos em qualquer repositório na sua máquina, você deve utilizar quais comandos? A) `git config --global user.name "Nome"` e `git config --global user.email "email@exemplo.com"` B) `git setup name="Nome" email="email@exemplo.com"` C) `git commit --user="Nome" --email="email@exemplo.com"` D) `git auth login --name "Nome" --email "email@exemplo.com"` E) `git init --profile "Nome" "email@exemplo.com"`

QUESTÃO 27 (Peso 2)

O que é o GitHub Actions? A) Uma funcionalidade para criar sites estáticos diretamente de um repositório. B) Uma plataforma de CI/CD (Integração e Entrega Contínuas) integrada ao GitHub que permite automatizar pipelines de testes, builds e deploys. C) O sistema de curtidas e interações (ações) nas Issues e Pull Requests. D) Um aplicativo desktop do GitHub para gerenciar repositórios visualmente. E) Um comando do Git para executar scripts Python automaticamente antes do commit.

QUESTÃO 28 (Peso 3)

Você adicionou um arquivo sensível (ex: `.env`) ao rastreamento do Git por engano e ele já foi comitado, mas você não quer apagá-lo do seu computador, apenas removê-lo do rastreamento do Git para os próximos commits. Qual o comando adequado? A) `git delete .env` B) `git rm --cached .env` C) `git reset --hard .env` D) `git stash pop .env` E) `git ignore add .env`

QUESTÃO 29 (Peso 2)

Quando ocorre um conflito de mesclagem (Merge Conflict), o Git insere marcadores diretamente no arquivo para que você escolha qual código manter. Qual marcador indica o início do código da branch em que você está atualmente (HEAD)? A) >>>>>>

origin/branch B) ===== C) <<<<<< HEAD D) ||||| merged common
ancestors E) ----- CURRENT

QUESTÃO 30 (Peso 3)

O comando `git bisect` é uma ferramenta poderosa para depuração (debugging). Qual é a sua utilidade central? A) Dividir repositórios grandes (monorepos) em múltiplos repositórios menores. B) Aplicar busca binária no histórico de commits para encontrar rapidamente o commit exato que introduziu um bug. C) Comparar os commits de duas branches diferentes e gerar um relatório de sobreposição. D) Corrigir automaticamente os marcadores de conflitos durante um rebase complexo. E) Desfazer os últimos dois commits simultaneamente.

QUESTÃO 31 (Peso 1)

Você guardou o seu progresso inacabado diversas vezes seguidas usando o `git stash`.

Como você pode visualizar a lista de todos os stashes armazenados para escolher qual

aplicar? A) `git stash list` B) `git stash show --all` C) `git log --stash` D) `git status --stored` E) `git show stash`

QUESTÃO 32 (Peso 2)

Ao enviar uma branch local para o repositório remoto pela primeira vez, utiliza-se a flag `-u`

(ou `--set-upstream`) em `git push -u origin minha-branch`. Qual o objetivo prático dessa flag? A) Forçar a sobrescrita do histórico remoto, ignorando as proteções de branch. B) Criar um link permanente de rastreamento entre a branch local e a remota,

permitindo que usos futuros de `git push` e `git pull` nessa branch não exijam argumentos adicionais. C) Exigir que o envio seja autenticado exclusivamente via token de acesso pessoal. D) Impedir que outros colaboradores possam fazer push na mesma branch remotamente. E) Unificar todos os commits locais em um único commit antes de enviar ao servidor.

QUESTÃO 33 (Peso 2)

O que é um "Fast-forward merge" no Git? A) Um merge que falha automaticamente se houver mais de 5 arquivos modificados. B) Um processo que ocorre quando a branch de destino não possui commits divergentes após a criação da branch de origem, permitindo que o Git apenas avance o ponteiro HEAD sem criar um "merge commit". C) O comando usado no GitHub para acelerar o download do repositório em conexões lentas. D) Uma técnica que une apenas as mudanças textuais (commits) e descarta as mudanças em

imagens ou binários. E) Um rebase disfarçado que reescreve todo o histórico do projeto e altera as chaves criptográficas (hashes).

QUESTÃO 34 (Peso 2)

Após executar vários testes localmente, o seu Working Directory está cheio de arquivos gerados (logs, builds temporários) que não são rastreados (untracked) e não estão no `.gitignore`. Qual comando remove permanentemente todos esses arquivos não rastreados de uma só vez? A) `git rm -rf .` B) `git reset --hard HEAD` C) `git clean -fd` D) `git restore .` E) `git garbage --collect`

QUESTÃO 35 (Peso 1)

No GitHub, a funcionalidade "Releases" é utilizada primordialmente para: A) Agendar a publicação automática de novos Pull Requests. B) Distribuir pacotes empacotados, binários de software compilado ou notas de versão associadas a uma Tag do Git. C) Excluir o histórico antigo do repositório para liberar espaço. D) Transformar um repositório privado em público temporariamente. E) Impedir edições em arquivos específicos da branch principal.

QUESTÃO 36 (Peso 2)

Para examinar detalhadamente um commit específico, verificando sua mensagem, autor, data e as diferenças exatas (diff) das modificações que ele introduziu, qual comando você deve usar? A) `git inspect <hash>` B) `git log --diff <hash>` C) `git show <hash>` D) `git status --commit <hash>` E) `git blame <hash>`

QUESTÃO 37 (Peso 3)

Durante um `git cherry-pick`, ocorreu um conflito. Após você editar os arquivos e resolver o conflito, qual a sequência correta para concluir o processo de cherry-pick? A) `git add <arquivos>` e `git cherry-pick --continue` B) `git commit` e `git cherry-pick --finish` C) `git push` e `git cherry-pick --resolve` D) `git stash` e `git pop` E) `git add <arquivos>` e `git merge --continue`

QUESTÃO 38 (Peso 3)

Você trabalhou em uma nova funcionalidade e gerou 10 pequenos commits (ex: "ajuste", "correção de erro", "wip"). Antes de integrar à branch principal ou abrir um Pull Request, você deseja unificar todos eles em um único e limpo commit. Qual é a prática recomendada no Git para isso? A) Executar `git merge --squash` na própria branch. B) Fazer um Rebase interativo (`git rebase -i`) e utilizar a opção `squash` (ou `s`) para aglutinar os commits. C) Rodar `git revert` 10 vezes e fazer um novo commit. D) Usar o comando nativo `git compress --all`. E) Criar uma Tag e deletar a branch original.

QUESTÃO 39 (Peso 2)

Você executa o comando `git branch -a` e vê diversas branches remotas listadas como `remotes/origin/feature-xyz` que, na verdade, já foram deletadas do GitHub pelos seus colegas após o merge dos Pull Requests. Como você limpa o seu registro local dessas referências remotas obsoletas? A) `git remote clean origin` B) `git fetch --prune` C) `git branch --clear-remotes` D) `git push origin --purge` E) `git rebase --clean`

QUESTÃO 40 (Peso 3)

Embora o `git reflog` atue como uma "rede de segurança" local registrando os movimentos da HEAD, essas informações não são mantidas para sempre. Qual das alternativas reflete corretamente o comportamento de expiração do Git em relação às entradas do reflog inalcançáveis? A) Elas são mantidas indefinidamente até o sistema operacional reiniciar. B) Elas são excluídas em 24 horas por motivos de segurança criptográfica. C) O Git as remove permanentemente a cada vez que um `git push` é executado. D) Por padrão, entradas inalcançáveis (como as criadas após um rebase reescrito) expiram e podem ser coletadas pelo Garbage Collector do Git em cerca de 30 dias. E) O reflog sincroniza com o GitHub mensalmente, limpando os registros locais de acordo com os limites da conta.

Gabarito – Simulado 2/8 (Git e GitHub)

1 a 10

- 01. B (O arquivo fica tanto no Staging quanto no Working Directory)
- 02. C (O `.gitignore` não afeta arquivos que já estão sendo rastreados)
- 03. B (`pull` = `fetch` + `merge`)
- 04. B (O HEAD aponta para um hash e não para o nome da branch)
- 05. C (Resolver no código, salvar, dar `add` e depois `commit`)
- 06. B (Ferramenta de colaboração/revisão de código no GitHub)
- 07. B (`git commit --amend` junta a alteração ao último commit)
- 08. A (`pop` aplica e descarta do stash; `apply` aplica mas mantém listado)
- 09. B (Rebase cria histórico linear; Merge cria commit de mesclagem)
- 10. C (Estando na destino, fazemos merge na origem)

11 a 20

- 11. A (Cria repo, baixa arquivos, cria branch e linca o origin)
- 12. B (`reflog` guarda todo o histórico de movimentos do HEAD)
- 13. B (Cópia do repo na sua conta para alterar livremente)
- 14. A (`git remote -v` mostra os links)
- 15. C (`cherry-pick` pesca um commit específico de outra branch)
- 16. B (Histórico resumido com hash curto e mensagem em uma linha)

- 17. C (*git revert* cria um novo commit desfazendo o antigo, mantendo o histórico)
- 18. B (*git push origin --delete <branch>* exclui no remoto)
- 19. B (Marca versões/releases de forma permanente)
- 20. A (*git diff* vê alterações que ainda não foram para o stage)

21 a 30

- 21. B (Arquivo *.gitkeep* dentro da pasta)
- 22. B (*--mixed* mantém as edições na máquina; *--hard* apaga tudo)
- 23. B (Rastreador de tarefas, bugs e discussões nativo do repositório)
- 24. C (*git remote add upstream <url>* linca o repo original)
- 25. B (Não inclui arquivos untracked/novos)
- 26. A (*git config --global user.name* e *user.email*)
- 27. B (Plataforma de automação/CI-CD integrada)
- 28. B (*git rm --cached* tira do rastreo mas mantém o arquivo na máquina)
- 29. C (*<<<<<<< HEAD* marca o código atual)
- 30. B (Busca binária para achar o commit que introduziu o erro)

31 a 40

- 31. A (*git stash list*)
- 32. B (*--set-upstream* linka as branches para futuros push/pulls diretos)
- 33. B (Avança o HEAD numa linha reta sem precisar de commit de mesclagem)
- 34. C (*git clean -fd* limpa arquivos e diretórios untracked)
- 35. B (Distribuição de pacotes e notas baseadas em Tags)
- 36. C (*git show <hash>*)
- 37. A (*add* e depois *git cherry-pick --continue*)
- 38. B (Rebase iterativo com a opção *squash* junta os commits)
- 39. B (*git fetch --prune* limpa o rastreo local de branches deletadas no remoto)
- 40. D (Entradas inalcançáveis são limpas pelo Garbage Collector em cerca de 30 dias)